

Classification Models – Theory and Application

Hoang Trung Dung*, Le Minh Kiet*, Le Tam Quang*,
Nguyen Vu Thuy*, Nguyen Tat Hung*

Hanoi University of Science and Technology

*These authors contributed equally. The order of authorship is arbitrary.

June 6, 2025

Abstract

This report explores the mathematical foundations of five popular classification models - logistic regression, support vector machines, naive Bayes, decision trees, and random forests. In addition to the theoretical deep dive, the authors also implement each model in code using a well-known benchmark dataset that has been widely used for evaluating classification algorithms. The authors' goal is to understand these models both conceptually and practically, and to compare their performance on real-world data. Overall, the models demonstrated similar performance when predicting the dominant class, largely due to class imbalance in the dataset. However, their performance dropped significantly when predicting the minority class. Among the five, the random forest model seemed to perform the best, achieving a higher F1 score than other models.

1 Introduction

This report presents a straight-forward exploration of five fundamental classification models using mathematical analysis and hands-on implementation. The team set out to understand these models not just at a surface level, but through a detailed study of their underlying mathematical principles. To bring theory into practice, a well-known and widely used dataset was chosen, and that will be described in detail later. This dataset serves as a benchmark for testing and comparing the performance of the models.

The structure of this report is as follows: First, we explain the workings of each classification method, which are the mathematical foundations that support them. Next, we describe the dataset, including its features and preprocessing. Finally, we present the results of applying the models to the dataset.

2 Analysis Methods

2.1 Logistic Regression

This section will introduce logistic regression. In the first part, there will be a high level overview of the method. The second part provides the mathematical formulation of logistic regression. Finally, there will be a simple solved example in the last part.

2.1.1 Log! What Is It Good For?

To understand why we have logistic regression and what it is good at, we need to talk about linear regression first. Linear regression is very simple. It is simple to understand and visualize. It is simple to work out mathematically as well [2]. It is also very powerful and capable, witnessing application across many fields, from modeling GDP in finance [4], to managing pollution in environmental science [3], and is a foundational technique in machine learning.

However, linear regression can only model data that is, well, linear, and has a very large domain generally. It is not a very effective tool to model, let's say, binary data. This is because binary data is not continuous, and has a very limited domain of 0 and 1. Even when we relax the requirement and try to model the probability of getting certain

binary results (for example, the probability of having lung cancer, or not), linear regression still isn't a very good fit (Figure 1). Compare that with logistic regression (Figure 2).

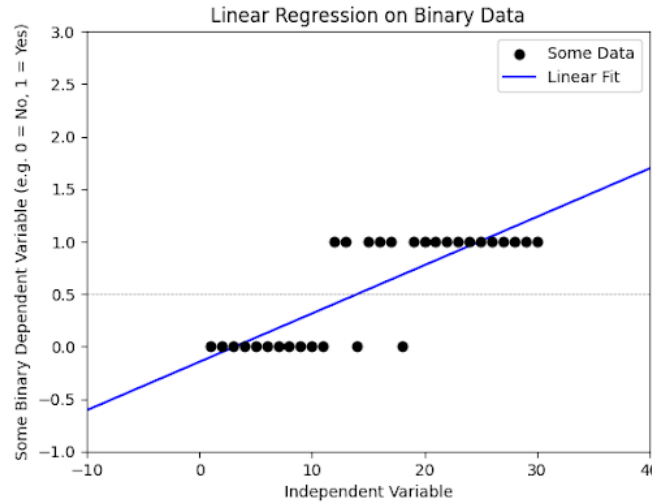


Figure 1: Fitting a line to some binary data. This is not a good model.

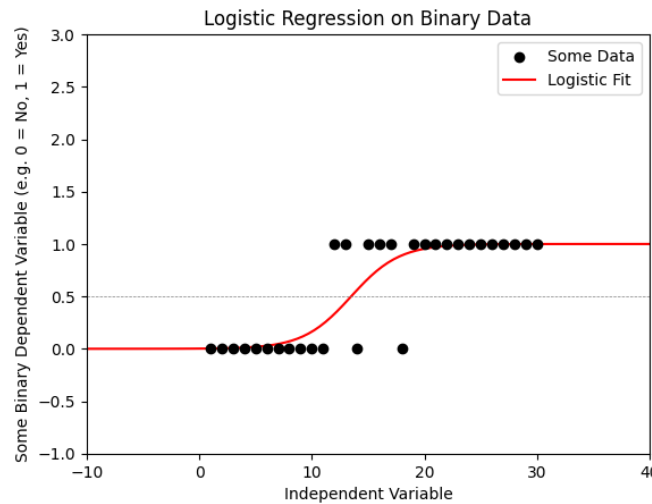


Figure 2: Using a logistic curve (also called a sigmoid function) to fit our binary data. We can see that this is a much better fit.

Logistic regression is able to model the probabilities of getting some binary data in a much better way than linear regression, as evident in the figures above. Furthermore, it still inherits the simplicity of linear regression at its core. This is why logistic regression is a prime option for the dataset we have got at hand.

2.1.2 How It's Made, Mathematically

Here's the logic: We want to model the value of an indicator Y , which is always 1 or 0 - this is how we turn something qualitative into something quantitative. Unfortunately, a model of the form $\beta_0 + \beta_1 X_1 + \beta_2 X_2$ doesn't produce values that are exactly 0 or 1. Thus, instead we focus on modeling the probability that $Y = 0$ or $Y = 1$. Even this isn't quite good enough as a probability is between 0 and 1 and there is no certainty that $\beta_0 + \beta_1 X_1 + \beta_2 X_2$ will lie in that range. Thus, we do one more trick which is to take the odds that $Y = 1$, given by $\frac{P(Y=1)}{P(Y=0)}$, and take the log to get a number between negative and positive infinity. This is the number we model using our standard regression formula.

Here's what all of it looks like mathematically:

$$\log \left(\frac{P(Y=1)}{P(Y=0)} \right) = \beta_0 + \beta_1 X_1 + \beta_2 X_2$$

Note that $P(Y=0)$ is actually just equal to $1 - P(Y=1)$, and that this function is called a logit function. Next, to solve for all the parameters and get a useful model, we need to break this down and get an analytic form for $P(Y=1)$:

$$P(Y=1) = \frac{1}{1 + e^{-(\beta_0 + \beta_1 X_1 + \beta_2 X_2)}}$$

This is the aptly named “logistic function”, which we will denote as $p(x)$ from now on.

And then we construct the likelihood function for the parameters with all the given datapoints:

$$L(\beta) = \prod_{i \text{ where } y_i=1} p(x_i) \prod_{i \text{ where } y_i=0} (1 - p(x_i))$$

We're multiplying over all the predicted probabilities $p(x_i)$ for all samples where $y_i = 1$, and the complement probabilities $1 - p(x_i)$ for all samples where $y_i = 0$, and we want this product to be as close to 1 as possible. We can express this in a more compact and arguably smarter way:

$$L(\beta) = \prod_{i=1}^n p(x_i)^{y_i} (1 - p(x_i))^{1-y_i}$$

We're making use of the binary nature of the variable y . Since y_i can only be either 0 or 1, the expression $p(x_i)^{y_i} (1 - p(x_i))^{1-y_i}$ automatically selects the correct term: if $y_i = 1$, the second term becomes 1; if $y_i = 0$, the first term becomes 1! But we are not stopping here. Products are cumbersome and more difficult to work with, so we'll convert the likelihood function into the log-likelihood function:

$$\ell(\beta) = \ln L(\beta) = \sum_{i=1}^n [y_i \ln p(x_i) + (1 - y_i) \ln(1 - p(x_i))]$$

and then we can substitute all the $p(x_i)$ for the full expression to obtain our final function of β :

$$\ell(\beta) = \sum_{i=1}^n \left[y_i (\beta_0 + \beta_1 x_i) - \ln \left(1 + e^{-(\beta_0 + \beta_1 x_i)} \right) \right]$$

Our goal now is to find the parameters β that makes the log-likelihood as close to 0 as possible. Originally, we aimed make the likelihood very close to 1, but after taking the logarithm, we now want to make the log-likelihood 0 since $\ln(1) = 0$. This equation is a transcendental equation, which cannot be solved directly in a closed-form solution. So we must resort to approximation methods, like Gradient Descent or Newton-Raphson Numerical Method

2.1.3 A Solved Example

Table 1: Dataset

ID	Cigarettes Per Day	Lung Cancer
1	0	No
2	2	No
3	5	No
4	7	Yes
5	10	No
6	12	Yes
7	15	Yes
8	18	Yes
9	20	Yes
10	25	Yes

Given the following very simple dataset, we are now to model the data using logistic regression. First, let x_1 represent **Cigarettes Per Day**, and let x_2 represent **Lung Cancer** (0 = No, 1 = Yes).

The probability of getting lung cancer will now look like this:

$$P(\text{Lung Cancer} = 1) = \frac{1}{1 + e^{-(\beta_0 + \beta_1 x_1)}}$$

Then we can construct the likelihood function as usual and get the following log-likelihood function:

$$\ell(\beta) = \sum_{i=1}^n \left[y_i (\beta_0 + \beta_1 x_i) - \ln \left(1 + e^{-(\beta_0 + \beta_1 x_i)} \right) \right]$$

And we expand the sum:

$$\begin{aligned} \ell(\beta) = & 0(\beta_0 + \beta_1 \cdot 0) - \ln(1 + e^{-(\beta_0 + \beta_1 \cdot 0)}) \\ & + 0(\beta_0 + \beta_1 \cdot 2) - \ln(1 + e^{-(\beta_0 + \beta_1 \cdot 2)}) \\ & + 0(\beta_0 + \beta_1 \cdot 5) - \ln(1 + e^{-(\beta_0 + \beta_1 \cdot 5)}) \\ & + 1(\beta_0 + \beta_1 \cdot 7) - \ln(1 + e^{-(\beta_0 + \beta_1 \cdot 7)}) \\ & + 0(\beta_0 + \beta_1 \cdot 10) - \ln(1 + e^{-(\beta_0 + \beta_1 \cdot 10)}) \\ & + 1(\beta_0 + \beta_1 \cdot 12) - \ln(1 + e^{-(\beta_0 + \beta_1 \cdot 12)}) \\ & + 1(\beta_0 + \beta_1 \cdot 15) - \ln(1 + e^{-(\beta_0 + \beta_1 \cdot 15)}) \\ & + 1(\beta_0 + \beta_1 \cdot 18) - \ln(1 + e^{-(\beta_0 + \beta_1 \cdot 18)}) \\ & + 1(\beta_0 + \beta_1 \cdot 20) - \ln(1 + e^{-(\beta_0 + \beta_1 \cdot 20)}) \\ & + 1(\beta_0 + \beta_1 \cdot 25) - \ln(1 + e^{-(\beta_0 + \beta_1 \cdot 25)}) \end{aligned}$$

Optimizing for a log-likelihood function as close to 0 as possible, we get the values of $\beta_0 = -4.0206$ and $\beta_1 = 0.4727$. Note that if we had normalized the values of cigarettes (by simply dividing all values by the maximum of 25), we would have obtained $\beta_0 = -3.9381$ and $\beta_1 = 11.6360$, which shows a clearer trend than what we got with the unnormalized values.

2.2 Support Vector Machines

Support Vector Machine (SVM) is a powerful and widely used supervised learning algorithm in machine learning, particularly suited for classification tasks. At its core, Support Vector Machine (SVM) works by finding the optimal boundary - called a hyperplane - that best separates data points belonging to different classes. This boundary is chosen so that it maximizes the margin, or distance, between the closest points of the classes and the hyperplane. These closest points are known as support vectors, and they play a crucial role in defining the position and orientation of the hyperplane.

SVM is a versatile algorithm capable of handling both linearly separable and non-linearly separable data. SVM can use the kernel trick to transform data into higher dimensions, making it possible to find a separating hyperplane even when the original data is not linearly separable. However, in this section, we will focus on the linear SVM, which assumes that the data can be separated by a straight line (or a flat hyperplane in higher dimensions).

In linear SVM, the goal is to find the best linear decision boundary that separates two classes of data points. There are two main variants of linear SVM, depending on how strictly the data must be separated:

1. The hard margin SVM is used when the data is perfectly linearly separable. It is highly sensitive to noise and outliers. Just a single misclassified point can make it impossible to find a separating hyperplane, making this approach impractical for most real-world data.
2. To overcome the limitations of the hard margin, the soft margin SVM allows some flexibility by permitting a small number of misclassifications or margin violations. It introduces a parameter (commonly denoted as C) that controls the trade-off between maximizing the margin and minimizing classification error. It is more suitable for real-world datasets that are not perfectly separable.

2.2.1 Hard Margin SVM

a. Primal problem

The goal of support vector machines (SVM) is to find a hyperplane $w^T x + b = 0$ that separates the classes while maximizing the minimum distance (margin) from the data points. For a training set $\{(x_i, y_i)\}_{i=1}^n$, where $x_i \in \mathbb{R}^d$ and $y_i \in \{\pm 1\}$, the distance from a point x_i to the hyperplane is given by $\frac{|w^T x_i + b|}{\|w\|}$ [13].

By scaling w and b such that the closest points satisfy $|w^T x_i + b| = 1$, the margin becomes $\frac{2}{\|w\|}$. Therefore, maximizing the margin is equivalent to minimizing $\|w\|$, subject to correct classification. Concretely, the primal optimization problem is:

$$\min_{w, b} \quad \frac{1}{2} \|w\|^2 \quad (1)$$

$$\text{subject to} \quad y_i(w^T x_i + b) \geq 1, \quad i = 1, \dots, n \quad (2)$$

Here, $\|w\|$ denotes the Euclidean norm. Each constraint $y_i(w^T x_i + b) \geq 1$ ensures that x_i lies on or outside the margin boundaries. Geometrically, the margin boundaries are defined by $w^T x + b = \pm 1$, and the support vectors (the closest data points) satisfy $y_i(w^T x_i + b) = 1$ [13].

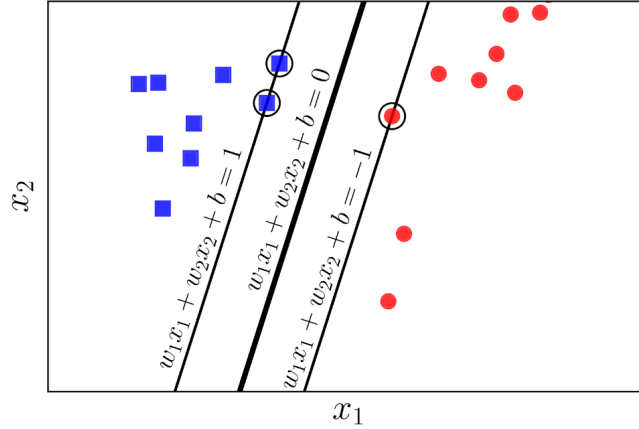


Figure 3: This figure illustrates a hard-margin support vector machine (SVM) in $\mathbb{R}^2$. The bold line represents the decision boundary defined by $w^T x + b = 0$, while the dashed lines denote the margin boundaries given by $w^T x + b = \pm 1$. Blue points correspond to the positive class ($y = +1$), and red points represent the negative class ($y = -1$). Only the support vectors—indicated as solid points on the margin boundaries—lie exactly at a perpendicular distance of $\frac{1}{\|w\|}$ from the decision boundary. Since the total margin width is $\frac{2}{\|w\|}$, minimizing $\|w\|^2$ serves to maximize this separation [7].

b. Dual problem

Introducing Lagrange multipliers $\alpha_i \geq 0$ for each constraint leads to the dual formulation of the SVM problem. By taking derivatives of the Lagrangian and setting them to zero, i.e., $\frac{\partial L}{\partial w} = 0$ and $\frac{\partial L}{\partial b} = 0$, one obtains the conditions:

$$w = \sum_{i=1}^n \alpha_i y_i x_i \quad \text{and} \quad \sum_{i=1}^n \alpha_i y_i = 0.$$

Substituting into the Lagrangian leads to the dual optimization problem [14]:

$$\begin{aligned} \max_{\alpha} \quad & \sum_{i=1}^n \alpha_i - \frac{1}{2} \sum_{i=1}^n \sum_{j=1}^n \alpha_i \alpha_j y_i y_j x_i^T x_j \\ \text{subject to} \quad & \sum_{i=1}^n \alpha_i y_i = 0, \quad \alpha_i \geq 0. \end{aligned} \quad (2)$$

The dual problem depends only on the inner products $x_i \cdot x_j$, which allows the use of kernels to implicitly operate in high-dimensional feature spaces. Solving the dual yields the same w as before [10]:

$$w = \sum_{i=1}^n \alpha_i y_i x_i.$$

Importantly, most α_i are zero—only the non-zero α_i correspond to the support vectors [11], i.e., points on the margin that actively define the classifier.

c. KKT Conditions

The Karush–Kuhn–Tucker (KKT) conditions ensure the equivalence between the primal and dual formulations of the support vector machine (SVM) optimization problem. For an optimal solution (w^*, b^*, α^*) , the KKT conditions include:

- Stationarity: Taking derivatives of the Lagrangian and setting them to zero gives [10]:

$$w^* = \sum_{i=1}^n \alpha_i^* y_i x_i \quad (3)$$

$$\sum_{i=1}^n \alpha_i^* y_i = 0 \quad (4)$$

- Primal feasibility:

$$y_i(w^\top x_i + b) \geq 1, \quad \forall i \quad (3)$$

- Dual feasibility:

$$\alpha_i^* \geq 0, \quad \forall i \quad (4)$$

- Complementary slackness:

$$\alpha_i^* [y_i(w^\top x_i + b) - 1] = 0, \quad \forall i \quad (5)$$

From complementary slackness (5), any point with $\alpha_i^* > 0$ lies exactly on the margin, i.e., satisfies $y_i(w^\top x_i + b) = 1$. These are the support vectors. Any point strictly outside the margin has $\alpha_i = 0$, meaning it does not affect the decision boundary [11].

d. Objective formula

After obtaining α from equation (2), one can deduce w from (3) and b from (5) and (4). It is clear that we only need to care about $\alpha_i \neq 0$.

Let $S = \{i : \alpha_i \neq 0\}$ and N_s be the number of elements in the set S . For each $i \in S$, we have:

$$1 = y_i(w^\top x_i + b) \iff b + w^\top x_i = y_i \quad (8)$$

Although for any pair (x_i, y_i) , one could immediately deduce b if one knew w , a different method for calculating b is often used and deduced to be more numerically stable:

$$b = \frac{1}{N_s} \sum_{i \in S} (y_i - w^\top x_i) = \frac{1}{N_s} \sum_{i \in S} \left(y_i - \sum_{j \in S} \alpha_j y_j x_j^\top x_i \right) \quad (9)$$

Previously, w was deduced to be:

$$w = \sum_{j \in S} \alpha_j y_j x_j \quad (10)$$

according to (3).

Observation: To determine which class a new point x belongs to, one needs to determine the sign of the expression [7]:

$$w^\top x + b = \sum_{j \in S} \alpha_j y_j x_j^\top x + \frac{1}{N_s} \sum_{i \in S} \left(y_i - \sum_{j \in S} \alpha_j y_j x_j^\top x_i \right) \quad (11)$$

2.2.2 Soft Margin SVM

a. Primal problem

When data are not perfectly separable, slack variables $\xi_i \geq 0$ allow margin violations. The soft-margin primal adds a penalty for misclassification:

$$\begin{aligned} \min_{w, b, \xi} \quad & \frac{1}{2} \|w\|^2 + C \sum_{i=1}^n \xi_i \\ \text{s.t.} \quad & y_i(w^T x_i + b) \geq 1 - \xi_i, \quad \xi_i \geq 0, \quad \forall i \end{aligned} \quad (6)$$

Here, $C > 0$ is a regularization parameter controlling the tradeoff: Larger C enforces fewer slack violations. Geometrically, some points are allowed within or beyond the margin at cost $C \cdot \xi_i$, but we still aim to keep them close to correct classification.

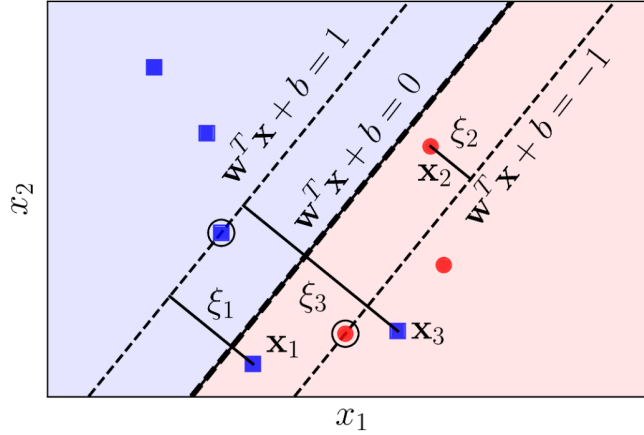


Figure 4: Illustration of the slack variables ξ_i . For points that lie in the safe region mathematically, $\xi_i = 0$. For points that lie in the unsafe region compared to the corresponding separating boundary, such as $0 < \xi_i < 1$, for example x_2 . For points that lie on the wrong side of the class compared to the separating boundary, such as $\xi_i > 1$, for example x_1 and x_3 [8].

b. Dual problem

The Lagrangian dual for the soft-margin SVM turns out to have the same quadratic objective as before, but with box constraints on α_i . One obtains:

$$\begin{aligned} \max_{\alpha} \quad & \sum_{i=1}^n \alpha_i - \frac{1}{2} \sum_{i=1}^n \sum_{j=1}^n \alpha_i \alpha_j y_i y_j x_i^T x_j \\ \text{s.t.} \quad & \sum_{i=1}^n \alpha_i y_i = 0, \quad 0 \leq \alpha_i \leq C \quad (i = 1, \dots, n) \end{aligned} \quad (7)$$

As detailed in standard SVM derivations, the dual is identical to the hard-margin case except that each α_i is upper-bounded by C . The solution again gives $w = \sum_{i=1}^n \alpha_i y_i x_i$ and remains sparse. The advantages of the dual are the same as before: only inner products appear (allowing kernels), and the sparsity ($\alpha_i > 0$ for only a few points) still holds.

c. KKT Conditions

At optimality $(w^*, b^*, \xi^*, \alpha^*, \mu^*)$, the KKT conditions [10] include:

- **Stationarity:**

$$w^* = \sum_{i=1}^n \alpha_i^* y_i x_i \quad \text{and} \quad \sum_{i=1}^n \alpha_i^* y_i = 0 \quad (8)$$

- **Primal feasibility:**

$$y_i(w^T x_i + b) \geq 1 - \xi_i^* \quad \text{and} \quad \xi_i^* \geq 0 \quad (9)$$

- **Dual feasibility:**

$$0 \leq \alpha_i^* \leq C \quad \text{and} \quad \mu_i^* \geq 0 \quad (10)$$

(where μ_i^* is the Lagrange multiplier for $\xi_i^* \geq 0$)

- **Complementary slackness:**

$$\alpha_i^*[y_i(w^T x_i + b) - 1 + \xi_i^*] = 0 \quad \text{and} \quad \mu_i^* \xi_i^* = 0 \quad (11)$$

These conditions imply the following familiar support-vector rules [5]:

- If $0 < \alpha_i^* < C$, then $\xi_i^* = 0$ and $y_i(w^T x_i + b) = 1$ (The point lies exactly on the margin).
- If $\alpha_i^* = C$, then $y_i(w^T x_i + b) \leq 1$ (The point is either on or inside the margin, possibly misclassified).
- If $\alpha_i^* = 0$, then $y_i(w^T x_i + b) \geq 1$ (The point lies strictly outside the margin).

d. Objective formula

Let $M = \{i : 0 < \alpha_i < C\}$ and $S = \{j : 0 < \alpha_j \leq C\}$. That is, M is the set of indices of points that lie exactly on the margins—or support the computation of b , and S is the set of indices of the support vectors—or support the computation of w . Similar to the case of Hard-Margin SVM, the parameters w and b can be determined as follows:

$$w = \sum_{j \in S} \alpha_j y_j x_j \quad (12)$$

$$b = \frac{1}{N_M} \sum_{i \in M} (y_i - w^T x_i) = \frac{1}{N_M} \sum_{i \in M} \left(y_i - \sum_{j \in S} \alpha_j y_j x_j^T x_i \right) \quad (13)$$

Note that the ultimate goal remains to determine the label for each new point, which does not have to be calculating w and b to quantify the optimal way to determine the values of the expression after substituting a new point x [8]:

$$w^T x + b = \sum_{j \in S} \alpha_j y_j x_j^T x + \frac{1}{N_M} \sum_{i \in M} \left(y_i - \sum_{j \in S} \alpha_j y_j x_j^T x_i \right) \quad (14)$$

2.3 Naïve Bayes

In our project, we wanted to find a probability-statistic based model for our predictions. Therefore, Naive Bayes is one of the most efficient and effective inductive learning algorithms for machine learning and data mining. This one has two main points. Firstly, it assumes that points are conditional independence, this means

$$P(X|C) = \prod_{i=1}^n P(X_i|C)$$

where $X = (X_1, X_2, \dots, X_n)$ is a feature vector and C is a class. The second part comes from Bayes Theory and leads to overall structure of this model. Moreover, one more important technique that should be included is Maximize A Posterior or MAP along with Maximize Likelihood Estimation. All above methods make Naïve Bayes more efficiently, reliability in real world problems field such as Text Classification. However, we attempt to research whether this is suitable for our purpose or not.

2.3.1 Background Theory

Let $\mathbf{X} = (X_1, X_2, \dots, X_n)$ be a vector of observed random variables, called features, where each feature takes values from its domain D_i . The set of all feature vectors is denoted $\Omega = D_1 \times \dots \times D_n$. Let C be an unobserved random variable denoting the class of an example, where C can take one of m values $c \in \{0, \dots, m-1\}$. Capital letters, such as X_i , will denote variables, while lower-case letters, such as x_i , will denote their values; boldface letters will denote vectors.

A function $g : \Omega \rightarrow \{0, \dots, m-1\}$, where $g(\mathbf{x}) = C$, denotes a concept to be learned. Deterministic $g(\mathbf{x})$ corresponds to a concept without noise, which always assigns the same class to a given example (e.g., disjunctive and conjunctive concepts are deterministic). In general, however, a concept can be noisy, yielding a random function $g(\mathbf{x})$.

A classifier is defined by a (deterministic) function $h : \Omega \rightarrow \{0, \dots, m-1\}$ (a hypothesis) that assigns a class to any given example. A common approach is to associate each class i with a discriminant function $f_i(\mathbf{x})$, $i = 0, \dots, m-1$, and let the classifier select the class with maximum discriminant function on a given example:

$$h(\mathbf{x}) = \arg \max_{i \in \{0, \dots, m-1\}} f_i(\mathbf{x}).$$

The Bayes classifier $h^*(\mathbf{x})$ (that we also call Bayes-optimal classifier and denote $BO(\mathbf{x})$), uses as discriminant functions the class posterior probabilities given a feature vector, i.e., $f_i^*(\mathbf{x}) = P(C = i \mid \mathbf{X} = \mathbf{x})$. Applying Bayes rule gives:

$$P(C = i \mid \mathbf{X} = \mathbf{x}) = \frac{P(\mathbf{X} = \mathbf{x} \mid C = i) \cdot P(C = i)}{P(\mathbf{X} = \mathbf{x})},$$

where $P(\mathbf{X} = \mathbf{x})$ is identical for all classes, and therefore can be ignored. This yields Bayes discriminant functions:

$$f_i^*(\mathbf{x}) = P(\mathbf{X} = \mathbf{x} \mid C = i) \cdot P(C = i),$$

where $P(\mathbf{X} = \mathbf{x} \mid C = i)$ is called the class-conditional probability distribution (CPD). Thus, the Bayes classifier

$$h^*(\mathbf{x}) = \arg \max_i P(\mathbf{X} = \mathbf{x} \mid C = i) \cdot P(C = i)$$

finds the maximum a posteriori probability (MAP) hypothesis given example $\mathbf{x}$. However, direct estimation of $P(\mathbf{X} = \mathbf{x} \mid C = i)$ from a given set of training examples is hard when the feature space is high-dimensional. Therefore, approximations are commonly used, such as using the simplifying assumption that features are independent given the class. This yields the naive Bayes classifier $NB(\mathbf{x})$ defined by discriminator functions:

$$f_i^{NB}(\mathbf{x}) = \prod_{j=1}^n P(X_j = x_j \mid C = i) \cdot P(C = i).$$

The probability of a classification error, or risk of a classifier h is defined as

$$R(h) = P(h(\mathbf{X}) \neq g(\mathbf{X})) = \sum_{\mathbf{x} \in \Omega} P(h(\mathbf{x}) \neq g(\mathbf{x})) \cdot P(\mathbf{X} = \mathbf{x}) = \mathbb{E}_{\mathbf{x}} \{P(h(\mathbf{x}) \neq g(\mathbf{x}))\},$$

where $\mathbb{E}_{\mathbf{x}}$ is the expectation over $\mathbf{x}$. $R^* = R(h^*)$ denotes Bayes error (Bayes risk).

When we research our dataset after processing (Figure 5), we realized that our dataset may work with naïve bayes, because some attributes are almost independent.

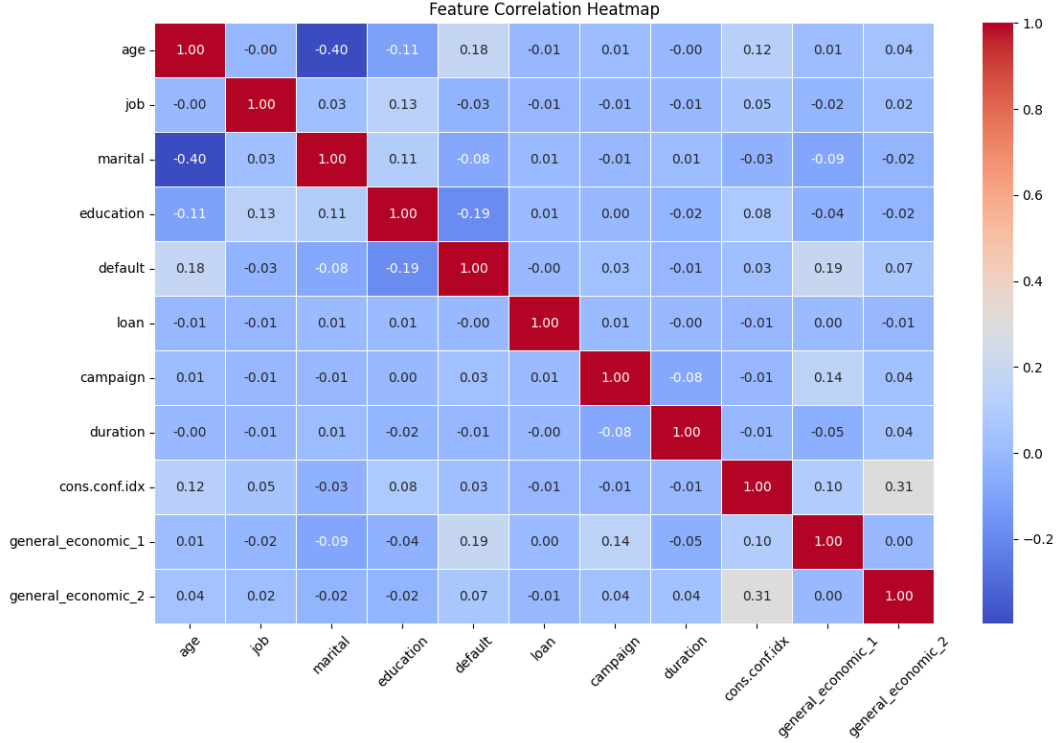


Figure 5: Feature correlation heatmap of the dataset

2.3.2 Gaussian Naïve Bayes

Gaussian Naive Bayes (GNB) is a probabilistic classifier that extends the classical Naive Bayes model to handle continuous-valued features by assuming they follow a Gaussian (normal) distribution. This means that

$$P(X = x_j | C = i) = \frac{1}{\sqrt{2\pi\sigma_{ji}^2}} e^{-\frac{(x_j - \mu_{ji})^2}{2\sigma_{ji}^2}}$$

where μ_{ji} , σ_{ji} denote the mean and variance of the feature j in class i , which are estimated from the training data using maximum likelihood estimation. For given class i and fixed feature j , we have

$$\mu = \frac{1}{n} \sum x_k \quad \forall x_k \in D_j \quad \text{and} \quad \sigma^2 = \frac{1}{n} \sum (x_k - \mu)^2$$

In our problem, we have classified two classes so that the Gauss Naïve Bayes defines

$$\text{Gnb}(\mathbf{x}) = \frac{f_1^{NB}(\mathbf{x})}{f_0^{NB}(\mathbf{x})} = \frac{\prod_{j=1}^n (P(X = x_j | C = 1) + \alpha) P(C = 1)}{\prod_{j=1}^n (P(X = x_j | C = 0) + \alpha) P(C = 0)} = \prod_{j=1}^n \text{Ps}(j) \times \text{Pc}$$

which

$$\text{Gnb}(\mathbf{x}) = \begin{cases} 1 & \text{if } h^*(\mathbf{x}) \geq 1 \\ 0 & \text{otherwise} \end{cases}$$

$$\text{Ps}(j) = \frac{\sigma_{j0}}{\sigma_{j1}} e^{\left(\frac{(x_j - \mu_{j0})^2}{2\sigma_{j0}^2} - \frac{(x_j - \mu_{j1})^2}{2\sigma_{j1}^2} \right)}, \text{ and Pc is ratio of two sample in each class}$$

α is added to avoid denominator being zero in case sample not appearing before.

2.4 Decision Tree

2.4.1 Introduction

A decision tree, a non-parametric supervised learning method, is one of the most popular machine learning models. This powerful tool makes predictions by learning from past examples and trying to organize the data into a tree-like structure. Each node is a decision, and the leaf assigns a class. By recursively splitting data into subsets based on attribute values, decision trees create clear, interpretable rules that help decision-makers predict outcomes, such as whether it will rain or if a medical procedure will succeed, making them widely applicable across various domains. As a consequence, it overcame limitations in other models like linear regression, logistic regression, discriminant analysis, and early rule-based systems.

2.4.2 Overview

In general, a decision tree classifies data items by generating several questions (split conditions) about the features associated with the items [6]. The questions can be either 'yes or no' (binary) or range-based, depending on the type of decision being evaluated. If that question can split the data into 2 sub-groups appropriately, that question will be stored in a node. The model keeps asking questions untiringly until the training data has been classified correctly, or the model has tested all the possible approaches. Figure 1 shows an example of a decision tree with its first 2 levels, where the first level splits the data by using the gender condition. The second level goes with the age range.

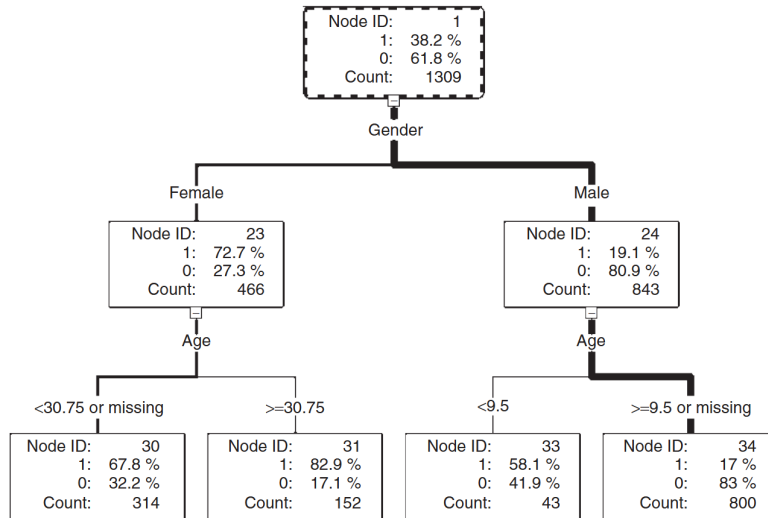


Figure 6: An example decision tree illustrating the analysis of survival in Titanic sinking with information on age and gender

Based on the structure, decision trees are sometimes more interpretable than other classifiers, such as neural networks and support vector machines, because they combine simple questions about the data in an understandable way [1]. Approaches for extracting decision rules from decision trees have also been successful. Unfortunately, small changes in input data can sometimes lead to large changes in the construction of trees. Decision trees are flexible enough to handle items with a mixture of real-valued and categorical features, as well as items with some missing features. They are expressive to model many partitions of the data that are not as easily achieved with classifiers that rely on a single decision boundary (such as logistic regression or support vector machines). However, even data that can be perfectly divided into classes by a hyperplane may require a large decision tree if only simple threshold tests are used. Decision trees naturally support classification problems with more than two classes. Finally, once constructed, they classify new items quickly.

2.4.3 Mathematical Foundation

The decision tree is constructed by generating various decision. At each node, the algorithm selects the best question that splits the data appropriately. These questions can be binary (e.g., "Is feature X $\leq$ a?") for continuous or categorical variables, or involve separating data into ranges. The quality of a split is determined by calculating the impurity of the resulting child nodes and selecting the split that maximally reduces impurity.

- For continuous features, thresholds are usually chosen by sorting the feature values and using the midpoints between consecutive values to evaluate.
- For categorical features, the split condition can be grouping categories or binary splits.

a. Gini impurity measurement

A widely used formula to calculate the impurity of each subgroup of a node is the Gini impurity [12]:

$$\text{Gini}(D) = 1 - \sum_{i=1}^k p_i^2$$

where D is the dataset at the node, k is the number of classes, and p_i is the proportion of instances in class i . At each node, the algorithm evaluates all possible splits for each feature (e.g., thresholds for continuous features or subsets for categorical features). The quality of a split is measured by the weighted impurity of the child nodes:

$$\text{Weighted impurity}(S) = \sum_{j \in \{L, R\}} \frac{|D_j|}{|D|} \times \text{Impurity}(D_j)$$

Here, D is the current node's dataset, D_j is the data in a child node j (where L is Left or R is Right), and $\frac{|D_j|}{|D|}$ is the proportion of data in child j . A lower weighted impurity indicates a better split, as it means the child nodes are more homogeneous. After calculating the weighted impurity of all variables in the dataset at the current node, the question with the lowest weighted impurity is chosen.

b. Early stopping and post pruning

While the decision tree is a powerful model, it faces overfitting when growing too complex and capturing noise or outliers in the dataset rather than general patterns. This occurs because the tree continues splitting nodes until it creates highly specific regions in the input space, often resulting in deep trees with many leaf nodes. To deal with that, some methods were proposed like early stopping and post pruning.

Early stopping, or pre-pruning, prevents overfitting by halting the tree's growth before it becomes too complex. This is achieved by setting constraints on the splitting process, such as:

- Maximum depth: Limiting the number of levels in the tree.
- Minimum node size: Requiring a minimum number of instances in a node to allow a split.
- Minimum impurity decreases: Only allowing splits that reduce impurity (e.g., Gini or entropy) by a specified threshold.

Early stopping helps reduce overfitting while speeding up the training process; however, it may lead to underfitting. If the stopping criteria are too strict, the tree might be too shallow to capture important patterns in the data, leading to underfitting.

Post pruning (cost-complexity pruning) involves growing a full, unrestricted decision tree and removing branches that contribute little to predictive accuracy. This is typically done by growing a full tree based on the training set, then evaluating the tree's performance on a validation set by using a cost-complexity metric, which balances tree size and error. The cost-complexity formula [9] is:

$$R_{\alpha}(T) = \text{Error}(T) + \alpha \cdot |T|$$

Where:

- $\text{Error}(T)$: The misclassification error of a subtree

- $|T|$: The number of leaf nodes in the tree
- α : A tuning parameter that controls the trade-off between error and complexity

The algorithm selects the subtree that minimizes $R_{\alpha}(T)$ for a given α . This is often done by testing multiple α values via cross-validation. Branches are pruned starting from the leaves, collapsing nodes into a single leaf if pruning reduces the cost-complexity score.

Post pruning provides better generalization while solving the overfitting issue. Since the tree is first grown fully and then pruned, post-pruning allows the model to consider all possible patterns before discarding irrelevant or weak patterns. But it is much slower and needs a validation set to determine which branches to prune, reducing the amount of data available for training the model.

2.5 Random Forest

Random Forest is a decision-tree based ensemble learning algorithm that combines multiple decision trees. Each single tree is trained via a random sample of the dataset with replacement (Bootstrap sampling) and a random subset of features (Feature selection). The forest combines the results of each one by majority voting method for classification tasks and averaging the result for regression tasks. This model overcomes the disadvantages of decision models (like overfitting, high variance, high bias, instability, ...) by utilizing the “wisdom of crowds” approach, making process tends to average out mistakes and arrive at more reliable predictions.

The Random Forest algorithm first constructs a finite number of trees (100 by default) and uses bootstrap sampling, feature selection on each one before growing all of them.

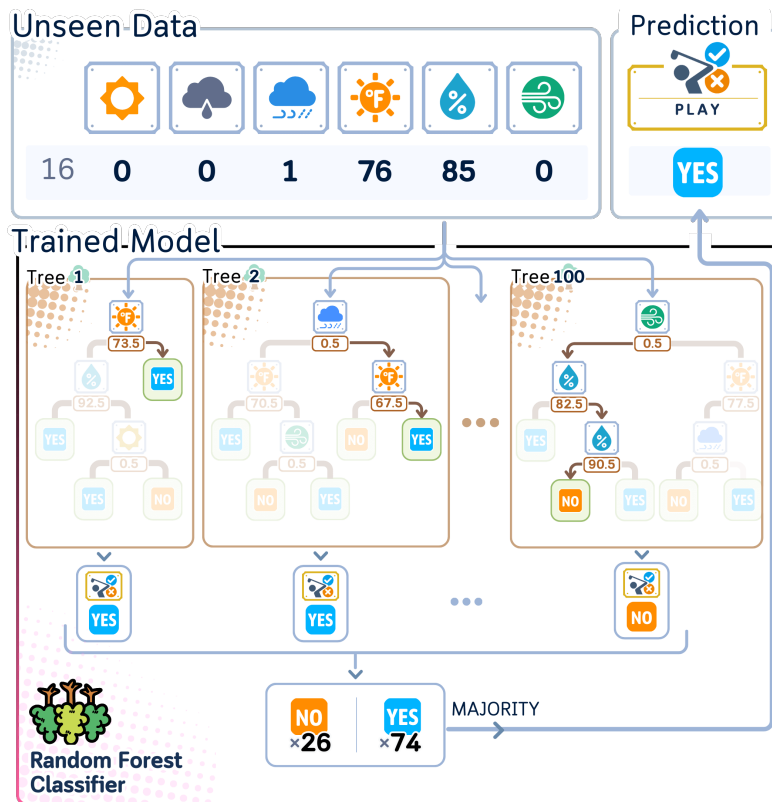


Figure 7: A Random Forest Classifier makes predictions by combining results from 100 different decision trees, each analyzing features like temperature and outlook conditions. The final prediction comes from the most common answer among all trees

2.5.1 Bootstrap Sampling

Each tree gets its own unique training set by sampling the original dataset with replacement. Records are allowed to appear multiple times in the sampled training sets, others may not be used.

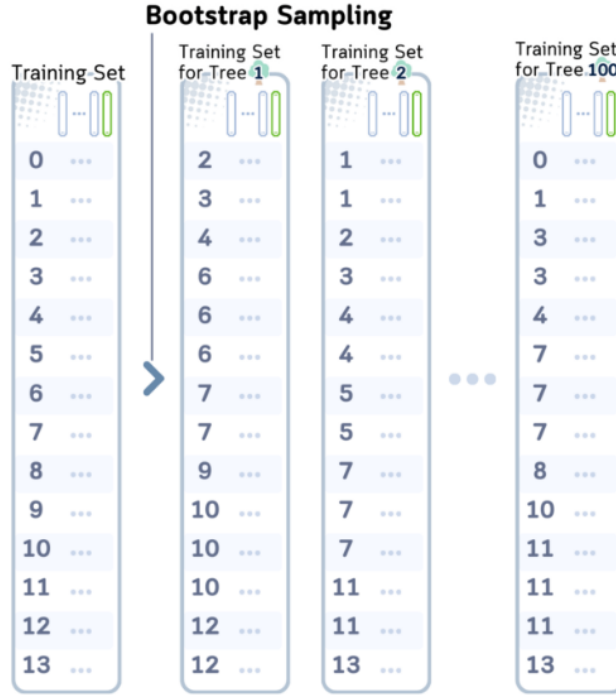


Figure 8: Illustration of the bootstrap sampling process (number of trees = 100)

2.5.2 Feature Selection

Each tree in the random forest only uses a subset of features in the original dataset. In classification tasks, the number of features used is often the square root of total features as it is proved to work well in many practical cases. Meanwhile, this number equals one third of the total features in regression tasks.

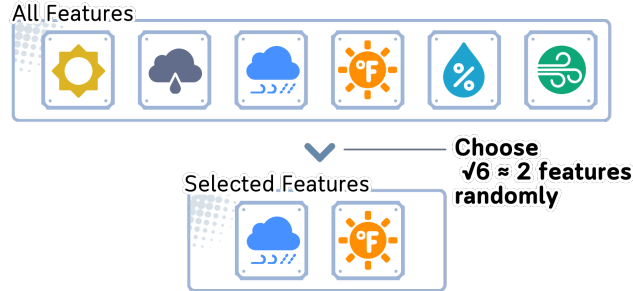


Figure 9: For each split in a tree, Random Forest randomly picks a subset of weather features (here 2 out of 6) to consider, making each tree focus on different aspects of the data

2.5.3 Computing Final Prediction

The first way is to use the Majority Voting Method, where we denote $T_i(x)$ as the prediction of the i -th tree. In classification problems, the final prediction is:

$$\hat{y} = \text{mode}\{T_1(x), T_2(x), \dots, T_n(x)\}$$

Another method is the Averaging Method. In regression problem the final prediction is given by:

$$\hat{y} = \frac{1}{n} \sum_{i=1}^n T_i(x)$$

2.5.4 Hyperparameters

There are three main hyperparameters in random forest algorithm: number of trees (100 on average), number of samples for each tree and number of features for each tree. To prevent overfitting and unstable cases, there is often a maximum depth limit set on each tree, along with the minimum samples required to split a node and minimum samples required at leaf nodes.

3 Dataset

3.1 Overview

For training and testing purposes, we found a small dataset on the internet. The Dataset called “The Bank Marketing” provides a comprehensive collection of customer-related and socio-economic attributes aimed at predicting client subscription behavior for term deposits. With 41,188 instances and 20 input variables, the dataset encompasses key demographic details (e.g., age, job, education), campaign-specific metrics (e.g., contact duration, previous outcomes), and macroeconomic indicators (e.g., employment rate, consumer confidence index). The objective of this dataset is determining whether the client had subscribed a term deposit, so the target “y” only consist of two values “yes” or “no”.

3.2 Exploratory Data Analysis (EDA)

So we attempted to investigate our data, to observe what the feature may effects final result (Figure 1). Consequently, dataset is imbalanced, the choice for denying term deposit is four times higher than the opposite one (88.7 % for rejection). However, it was not a worse problem for us, so we considered component rate in dataset and wiped out some features might be not useful for our prediction. For example, the record of age varied almost, especially peaking at age 31 and reducing in both sides of graph. Therefore, Age feature should be kept, similarity to Job, Marital and Education etc. In contrast, value in housing attribute except unknown is approximately like each other, this led us clear this attribute. The information above is attributed to customers.

3.3 Data Cleaning

Firstly, some categorical attributes have missing values, we solved this problem by filling all missing values with an ‘unknown’ value and recognized it as a part of feature. Additionally, from the graph, we observed that some attributes are right skew and outliers. For this problem, we applied Box plot method, this method simply focused on justifying the lower bound and upper bound of value in a feature for minimizing outliers. For example, we tried this method on recorded age (Figure 11) and duration feature (Figure 12).

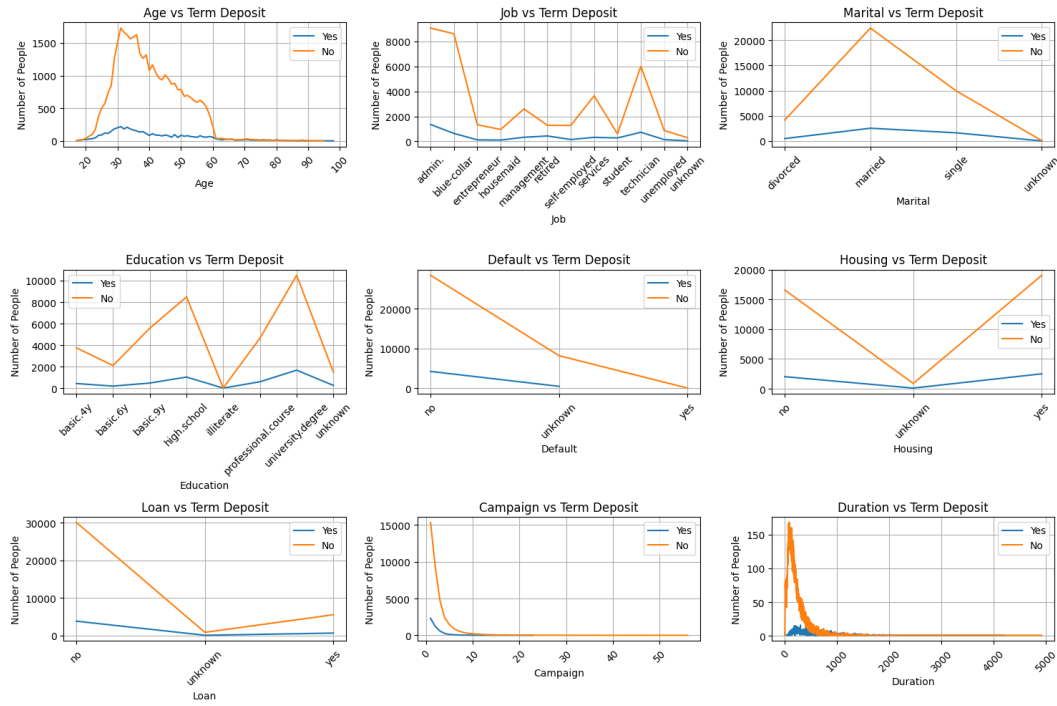


Figure 10: Different decision variables against term deposit decision

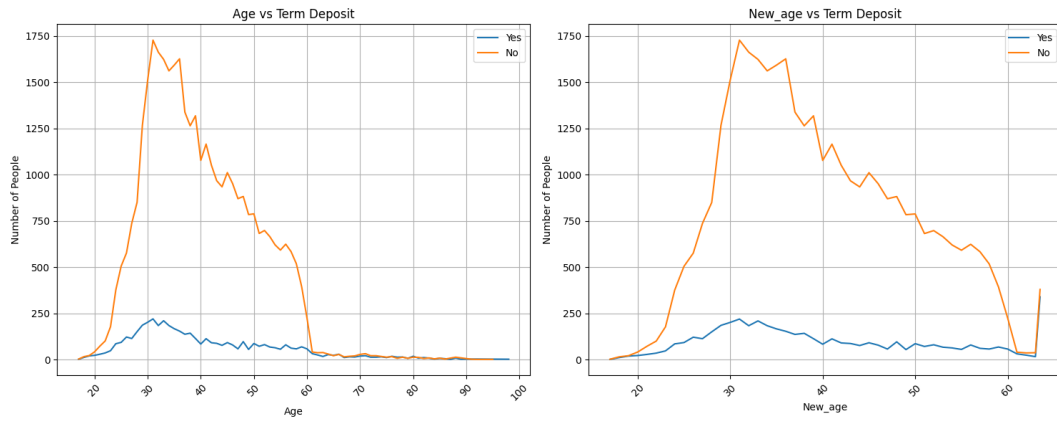


Figure 11: caption

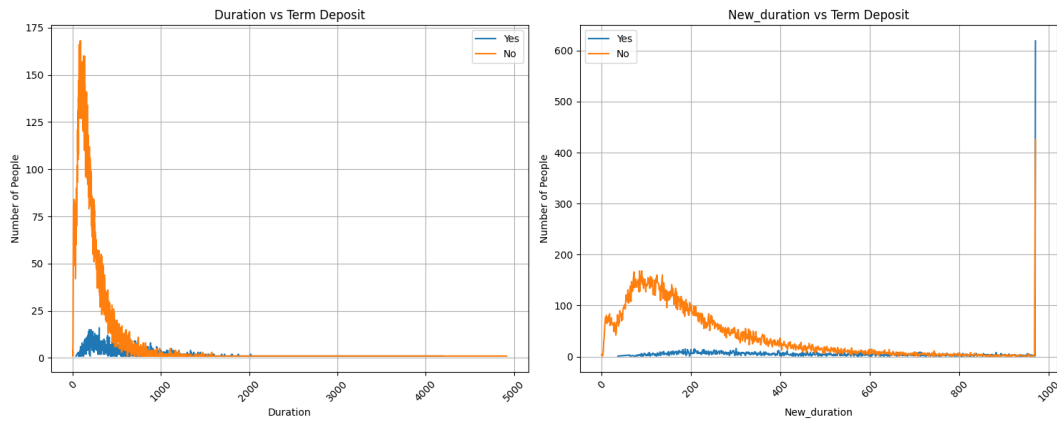


Figure 12: Null duration has been replaced with a value of nearly 1000

3.4 Feature Engineering

After cleaning data, we found out what features are important or relative to each other by using Heat map. As the graph shows (Figure 14), economic indexes except 'con.conf.idx' are highly correlated features, so we decided to use a simple dimension reduction called Principal Component Analysis (PCA) to compress these attributes to two components (Figure 13).

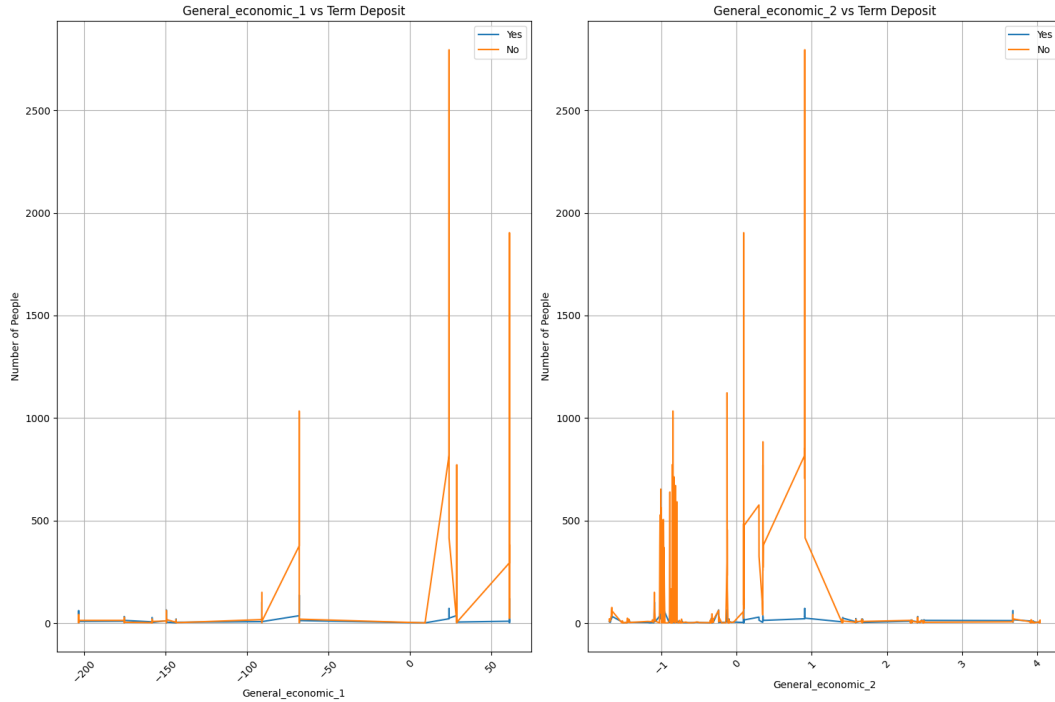


Figure 13: Principal Component Analysis (PCA) at work

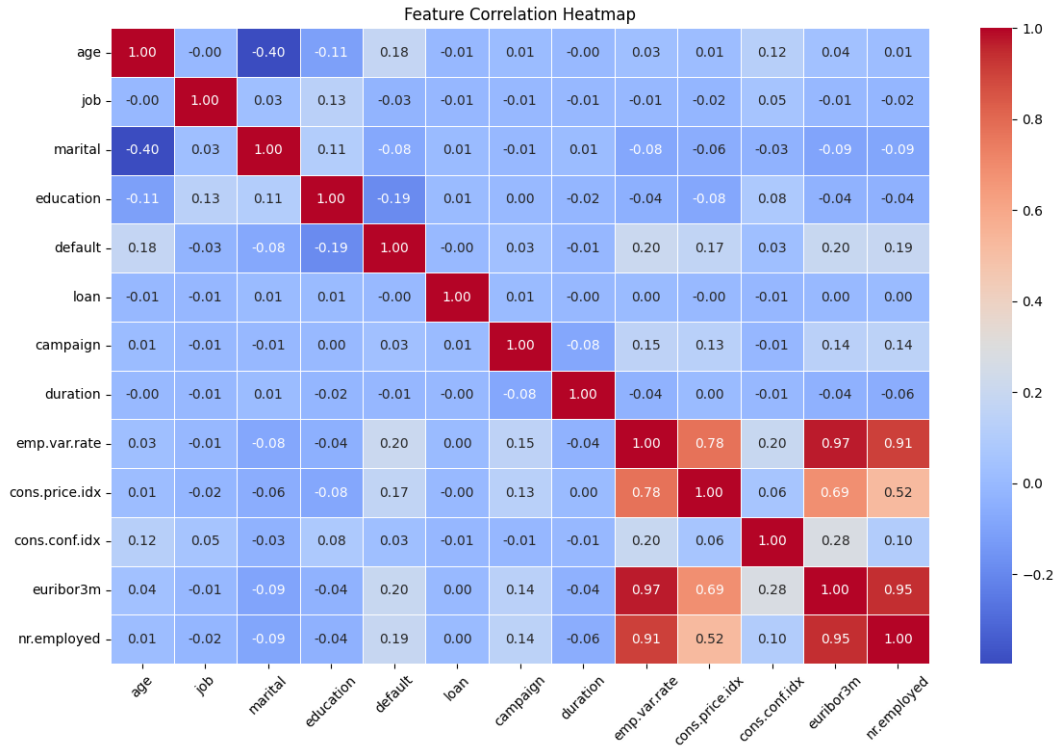


Figure 14: Some features are correlated

3.5 Data Preprocessing

3.5.1 SMOTE (Synthetic Minority Over-sampling Technique)

After researching about this dataset, we realize that we need learn how to handle significant imbalance data because value “no” of target variable ‘y’ make up 88.7%. The scenario of imbalance data may be misunderstood for training models, because models often predict ‘no’ more than ‘yes’. Therefore, we attempt to oversample our dataset to make them more balanced, SMOTE (Synthetic Minority Over-sampling Technique) is the most effective method corresponding to our dataset.

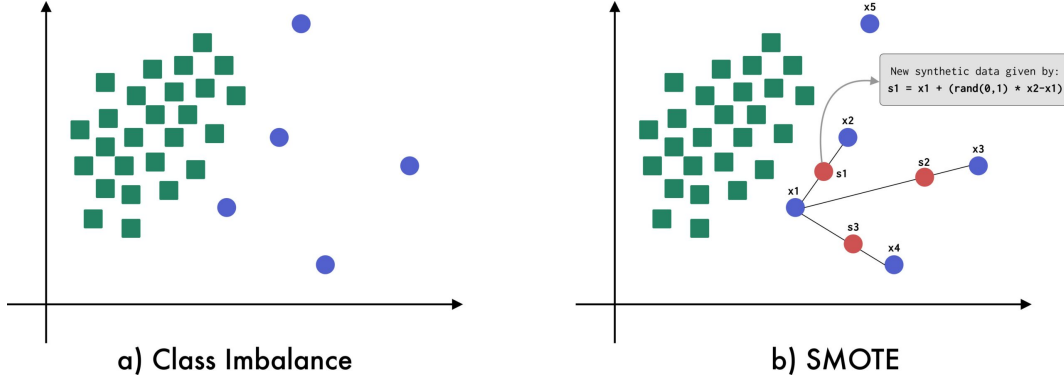


Figure 15: SMOTE as a potential solution to class imbalance problem

This technique concentrates on expanding the minority class by creating synthetic samples. Firstly, we need to determine what is minority class, then we select a point randomly in this class. Next, using K near neighborhood, we find k point nearest to chosen point, for each point, new artificial point will define

$$x_{\text{new}} = x_{\text{chosen}} + \lambda \cdot (x_{\text{nearest}_i} - x_{\text{chosen}})$$

With λ is random number in range $[0, 1]$. We do this until the number of all classes is the same. However, this method has some disadvantages like making some noises for data, this leads to dataset being more untrustworthy. Therefore, we use some variant methods from SMOTE like K-means SMOTE. This combines K-mean clustering and SMOTE to generate synthetic samples more intelligently. K-means clustering separate minority class into k clusters; and based on clusters that are high density and low distance to majority class, we apply SMOTE in each selected cluster.

K-means SMOTE consists of three steps: clustering, filtering, and oversampling. In the clustering step, the input space is clustered into k groups using k-means clustering. The filtering step selects clusters for oversampling, retaining those with a high proportion of minority class samples. It then distributes 5 synthetic samples to generate, assigning more samples to clusters where minority samples are sparsely distributed. Finally, in the oversampling step, SMOTE is applied in each selected cluster to achieve the target ratio of minority and majority instances. The algorithm is illustrated below.

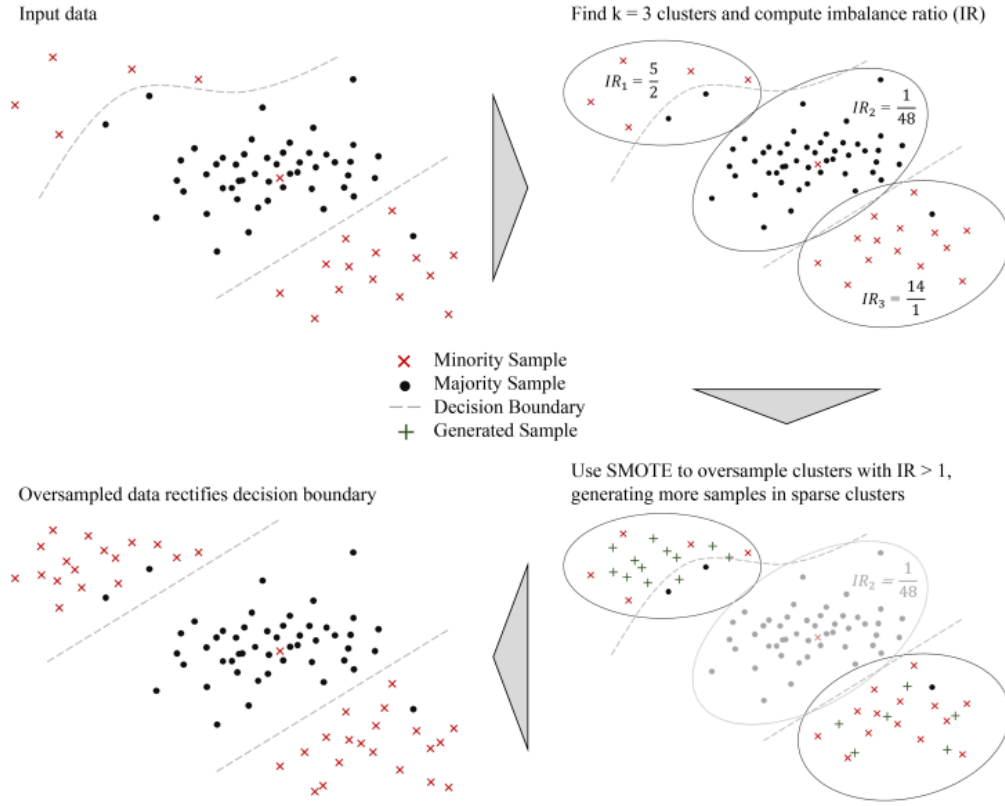


Figure 16: K-Means SMOTE diagrams

3.5.2 From SMOTE to our dataset

After attempting this approach, we realized that this may not be effective for our dataset because this could break the structure of data (Figure 17) and almost of our datasets have categorical attributes that make calculations more difficult. So, we chose simpler approach – Random Oversampling (Figure 18)

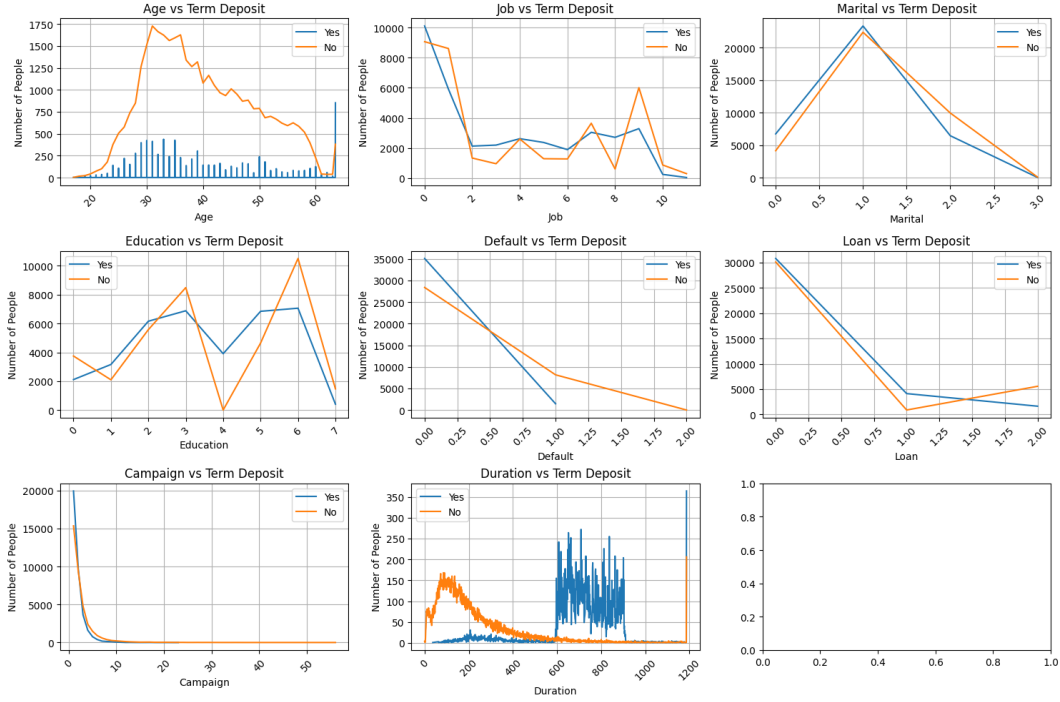


Figure 17: The structure of the data is broken

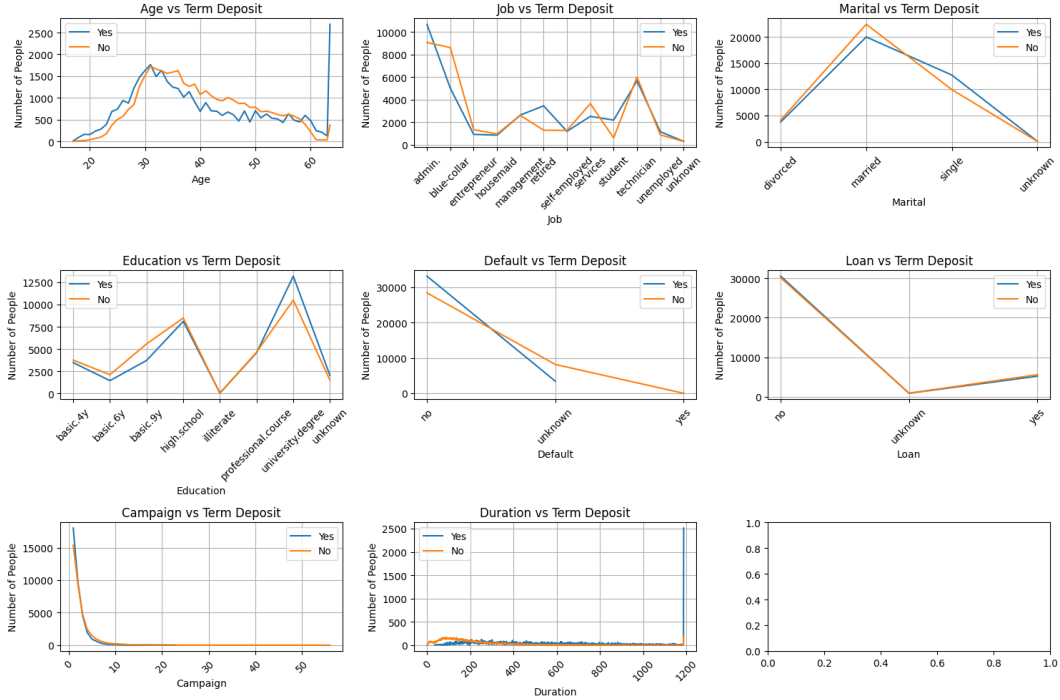


Figure 18: Random oversampling is a simpler and more effective approach

4 Results

Overall, all the models performed similarly when predicting class 0 (*No*). However, things get interesting with class 1 (*Yes*), where performance drops sharply for some models - down to half in certain cases. This likely stems from class imbalance in the dataset: there are significantly more *No* instances than *Yes*, which reflects real-world behavior - people are more likely to reject offers from the bank than accept them.

Precision is fairly consistent across SVM, random forest, and logistic regression, with roughly two-thirds of the *Yes* predictions actually being correct. In contrast, naive Bayes and decision trees correctly predict only about half of their *Yes* outputs. Looking over to recall, naive Bayes scores the highest here. It tends to label many more cases as *Yes*, which boosts recall at the expense of precision. On the opposite end, SVM and logistic regression are more conservative - they predict fewer *Yes* cases but are more accurate when they do. Among all this, random forest achieves the best overall balance: it has the highest precision, the second-highest recall, and consequently, the highest F1 score.

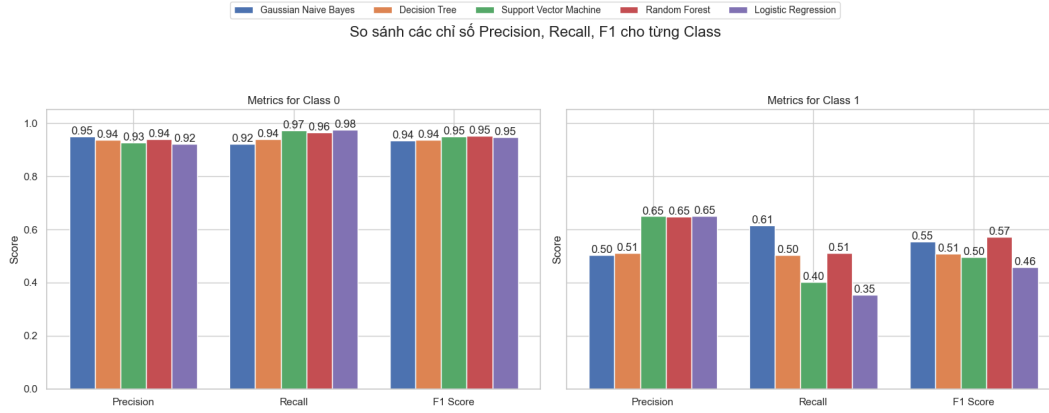


Figure 19: Model performances across class 0 (No) and class 1 (Yes)

The next figure is an interesting one. Overall, there are small bumps in performance when models are given preprocessed data. But a curious case is that of SVM, which saw its F1 score for class 1 prediction quite literally skyrocket from a mere 8% to 50%.



Figure 20: Comparisons of model performance before and after preprocessing

5 Member Contributions

- Hoang Trung Dung: Logistic regression.
- Le Minh Kiet: Naive Bayes.
- Le Tam Quang: Random forest.
- Nguyen Vu Thuy: Support vector machines.
- Nguyen Tat Hung: Decision trees.

References

- [1] B. de Ville. Decision trees. *WIREs Comput. Stat.*, 5:448–455, 2013.
- [2] Arun K. Kuchibhotla et al. All of linear regression. <https://arxiv.org/abs/1910.06386>, 2006.
- [3] S. Hoffman et al. Air quality modeling with the use of regression neural networks. <https://pmc.ncbi.nlm.nih.gov/articles/PMC9779138/>, 2023.
- [4] Zhaohao Fang. Application of linear regression in gdp forecasting. https://www.researchgate.net/publication/380849941_Application_of_Linear_Regression_in_GDP_Forecasting, 2024.
- [5] Michael I. Jordan. Soft margin svm. <https://people.eecs.berkeley.edu/~jordan/courses/281B-spring04/lectures/lec6.pdf>, 2004. Accessed: 2025-05-16.
- [6] C. Kingsford and S. L. Salzberg. What are decision trees? *Nature Biotechnology*, 26(9):1011–1013, 2008.
- [7] machinelearningcoba.com. Bài 19: Support vector machine. <https://machinelearningcoba.com/2017/04/09/smv/>, 2017. Accessed: 2025-05-16.
- [8] machinelearningcoba.com. Bài 20: Soft margin support vector machine. <https://machinelearningcoba.com/2017/04/13/softmarginismv/>, 2017. Accessed: 2025-05-16.
- [9] J. Mingers. An empirical comparison of pruning methods for decision tree induction. *Machine Learning*, 4:227–243, 1989.
- [10] David Rosenberg. Svm - complementary slackness. <https://davidrosenberg.github.io/mlcourse/Archive/2017/Lectures/5a.SVM-ComplementarySlackness.pdf>, 2017. Accessed: 2025-05-16.
- [11] MIT 6.034 Course Staff. Svm recitation notes. <https://web.mit.edu/6.034/wwwbob/svm-notes-long-08.pdf>, 2008. Accessed: 2025-05-16.
- [12] S. Tangirala. Evaluating the impact of gini index and information gain on classification using decision tree classifier algorithm. *Int. J. Adv. Comput. Sci. Appl. (IJACSA)*, 11(2), 2020.
- [13] Cornell University. Lecture 9: Svm. <https://www.cs.cornell.edu/courses/cs4780/2018fa/lectures/lecturenote09.html>, 2018. Accessed: 2025-05-16.
- [14] Ruye Wang. Norm soft margin. <https://pages.hmc.edu/ruye/e161/lectures/svm/node7.html>, 2016. Accessed: 2025-05-16.